

Planning, memory, and task framing

Browser Use: How LLM Agents Drive the Web

Week 8 Lecture 2

What we will cover

1. The planning layer: `enable_planning`, `PlanItem`, and when replanning fires
2. Procedural memory: the `memory` field in `AgentOutput` across steps
3. Task framing: the four properties of a task an agent can follow
4. Decomposing a hard task into sequenced sub-goals
5. The bitter-lesson argument: minimal loop, maximal action space

The planner

What the planner does

The planning layer generates and tracks an ordered list of `PlanItem` objects:

```
# PlanItem (from browser_use/agent/views.py)
class PlanItem:
    text: str
    status: Literal["pending", "current", "done", "skipped"]
```

The plan is stored in `AgentState.plan`.

The current item index is tracked in `AgentState.current_plan_item_index`.

When it fires:

- At the start of the run (initial plan generation)
- On stall: after `planning_replan_on_stall` consecutive failed steps

Enabling planning

```
from browser_use import Agent
from browser_use.llm import ChatAnthropic

agent = Agent(
    task=(
        "1. Go to news.ycombinator.com. "
        "2. Find the top story. "
        "3. Open the comments. "
        "4. Extract the three highest-voted comments. "
        "5. Return the text of each comment."
    ),
    llm=ChatAnthropic(model="claude-opus-4-8"),
    max_steps=60,
    max_failures=3,
)
```

Use planning for tasks with three or more clearly distinct stages.

planning_exploration_limit

`planning_exploration_limit` controls how many steps the agent can take within a plan item before the framework nudges it to update the plan.

This prevents the agent from drifting into off-plan exploration while still allowing flexibility within each stage.

After `planning_replan_on_stall` consecutive failures, the plan is regenerated from the current state. This is the planner's version of error recovery: not just retrying, but reconsidering the approach.

Procedural memory

The memory field

Every `AgentOutput` contains a `memory` field (from `browser_use/agent/views.py`):

```
class AgentBrain:
    evaluation_previous_goal: str # did the last step succeed?
    memory: str # notes the agent carries forward
    next_goal: str # what this step aims to do
```

`memory` is a string the agent writes to itself each step and reads back in the next step via the assembled messages.

It is not a database. It is a sticky note.

Why memory matters: the trim problem

`MessageManager` trims older browser-state summaries to fit the model's context budget (week 3).

An agent on step 30 cannot rely on scrolling back to step 3 to remember what it found.

The `memory` field is how information crosses the trim boundary:

- A price found on page 1 that is needed on page 5
- A login token extracted from a header
- Progress notes: "completed login, now on settings page"
- Failure context: "element 4 failed twice; trying search box"

Memory vs. message history

	memory field	Message history
Controlled by	The model	MessageManager
Survives trim	Always	Only if recent enough
You can inject content	No	Via system prompt
Appears in history	Yes, in every step	As messages

When you want information to survive across many steps, `memory` is more reliable than hoping a distant message survives the trim.

Task framing

Why task framing matters more than tuning

The task string is the agent's primary instruction.

A vague task produces an agent that wastes steps on ambiguity.

An overly rigid task produces an agent that fails when the page does not match expectations.

Four properties of a well-framed task:

1. A clear **starting point**
2. A clear **stopping condition**
3. Expected **scope**
4. A **failure hint**

Before and after

Poorly framed:

```
"Research the product and tell me what you find."
```

Better framed:

```
"Go to https://example.com/products/widget-x.  
Find the current price (it appears near 'Add to cart').  
If the price is not on the main page, check the 'Pricing' tab.  
Return the price as a number, e.g. '$29.99'.  
If you cannot find a price after checking both locations,  
return 'price not found'."
```

Each sentence maps to one of the four properties.

Decomposing a hard task

Number the stages explicitly in the task string:

```
task = (  
    "Complete these steps in order:\n"  
    "1. Go to example.com and log in with the credentials provided.\n"  
    "2. Navigate to Settings > Notifications.\n"  
    "3. Enable the 'Weekly summary' option.\n"  
    "4. Click Save.\n"  
    "5. Confirm the page shows a success message and return its text."  
)
```

Numbered steps give the agent a clear `next_goal` target.

They make `memory` easier to populate.

They make failures easier to diagnose: you can see which numbered step the agent was on when it failed.

Design philosophy

The bitter lesson applied to agents

"All the value is in the RL'd model, not your 10,000 lines of abstractions."

What this means for reliability

The bitter-lesson argument: general methods that scale with compute outperform human-crafted abstractions.

Applied to browser-use: the framework should be a minimal loop with a maximal action space.

Design recommendation: "Start with maximal capability, then restrict."

Give the agent the tools the task needs.

Add guards (`max_steps` , `max_failures` , loop detection) to prevent runaway behavior.

Do not add verification layers, output validators, and elaborate recovery logic if a better-framed task would solve the problem.

When to split into multiple agents

A single agent with 150 steps is harder to debug than three agents with 50 steps each.

Split into multiple sequential agents when:

- The task has stages with clearly different toolsets
- A failure in stage 2 should not re-run stage 1
- You want to inspect or validate the output between stages

The tradeoff: orchestration code and the overhead of launching separate runs.

This architectural decision is the focus of week 10 (capstone design).

Takeaways

- The planner generates a `PlanItem` list. It replans on stall. Use it for tasks with 3+ distinct stages.
- The `memory` field is the agent's sticky note across steps. It survives message-history trim.
- Task framing: starting point, stopping condition, scope, and failure hint.
- Numbered steps in the task string give the agent clear `next_goal` targets.
- The bitter lesson: a minimal loop with a maximal action space. Add guards, not layers.

What is next

Section (this week): take a flaky agent and task, diagnose from the history, and make it pass by reframing and adding guards

Week 9: evaluating agents, benchmarks, and the wider deployment ecosystem. The reliability work from this week feeds directly into measuring whether an agent is reliable enough to ship.